

语言程序设计

第十三讲 高级数据结构



张 华

主要内容

- 动态数据结构
- 自引用的结构体
- 动态内存分配
- 链表
- 程序设计举例

动态数据结构

■ Dynamic data structures (动态数据结构)

- ✱ 在程序的执行过程中可以增长和缩短的数据结构。

■ Linked lists (链表)

- ✱ 在任何地方进行插入和删除。

■ Stacks (栈)

- ✱ 在栈的顶端进行插入和删除。

■ Queues (对列)

- ✱ 在队列的末尾插入，在队列的前面删除。

自引用的结构体

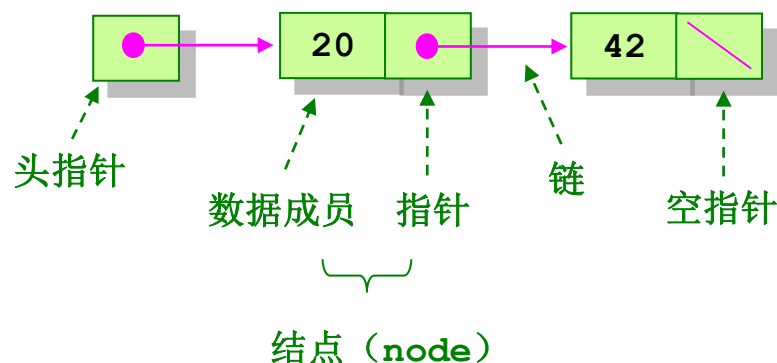
自引用的结构体

包含指向相同结构类型的指针成员

```
struct node {  
    int data;  
    struct node *nextPtr;  
};
```

可以链接到一起，构成有用的数据结构：

- 链表
- 栈
- 队列
- 树



动态内存分配

动态内存分配

- ✿ 在程序执行期间获取或释放内存

- ✿ 相关的函数 (`stdlib.h`)

 - `malloc()`

 - `free()`

- ✿ 相关的运算符

 - `sizeof`

动态内存分配

malloc()函数

void* malloc(size_t size)

✿ 参数**size**：表示要分配的内存的字节数

➤ **size_t**为**sizeof**返回的整数类型

➤ 一般用 **sizeof** 运算符确定对象的大小

✿ 返回值：指向分配内存的 **void*** 类型的指针

➤ **void*** 类型的指针可以被赋值给任何类型的指针变量

➤ 如果没有可用内存，则返回 **NULL**

```
int *array, n=30;  
array = (int*)malloc(n*sizeof(int));
```

```
struct node *sPtr;  
sPtr = malloc(sizeof(struct node));
```

动态内存分配

free()函数

void free(void* aPtr)

✱ 释放aPtr指向的内存空间

➤ aPtr指向的应该是由 malloc 分配的内存空间

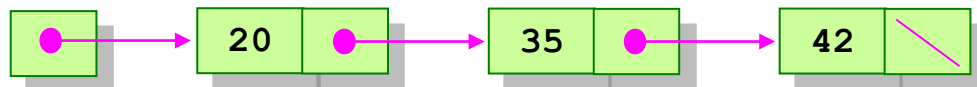
```
int *array, n=30;  
array = (int*)malloc(n*sizeof(int));  
...  
free(array);
```

➤ 如果不调用**free**释放**array**指向的空间，则程序结束后**array**消失了，但其所指的该段内存仍旧存在且不能被使用（其地址丢失了），即出现内存泄露。

链表

链表

✿ 用指针链接的自引用结构的线性集合，这些结构称为结点。



✿ 通过指向链表的第一个结点的指针访问链表。

✿ 后续的结点通过存储在每个结点中的链接指针成员来访问。

✿ 链表中最后一个结点的链接指针被设置为**NULL**，以标志链表的结尾。

什么时候用链表替代数组

✿ 数据成员的数目无法事先确定。

✿ 对数据进行快速的排序。

案例分析：链表

链表

问题

- 输入任意个整数创建链表。
- 实现常用的操作：遍历，插入，删除，空表判断。

实现 (cw1311.c)

```
#include <stdio.h>
#include <stdlib.h>

struct listNode {
    int data;
    struct listNode *nextPtr;
};

typedef struct listNode ListNode;
typedef ListNode* ListNodePtr;
```

案例分析：链表

链表

实现

```
void insert_node(ListNodePtr*, int);  
int delete_node(ListNodePtr*, int);  
int is_empty(ListNodePtr);  
void print_list(ListNodePtr);  
void menu(void);
```

案例分析：链表

链表

实现

```
void menu(void) {  
    printf("Enter your choice:\n"  
        "1 to insert an element into the list.\n"  
        "2 to delete an element from the list.\n"  
        "3 to end.\n");  
}
```

案例分析：链表

链表

实现

```
int main(void) {  
    ListNodePtr listPtr = NULL;  
    int choice;  
    int item;  
  
    menu();  
    printf("? ");  
    scanf("%d", &choice);
```

案例分析：链表

链表

实现

```
while (choice != 3) {  
    switch (choice) {  
        case 1:  
            printf("Enter an integer: ");  
            scanf("%d", &item);  
            insert_node(&listPtr, item);  
            print_list(listPtr);  
            break;
```

案例分析：链表

链表

实现

```
case 2:
    if (!is_empty(listPtr)) {
        printf("Enter an integer to be deleted: ");
        scanf("%d", &item);
        if (delete_node(&listPtr, item)) {
            printf("%d deleted.\n", item);
            print_list(listPtr);
        }
        else
            printf("%d not found.\n", item);
    }
    else
        printf("List is Empty!\n");
    break;
```

案例分析：链表

链表

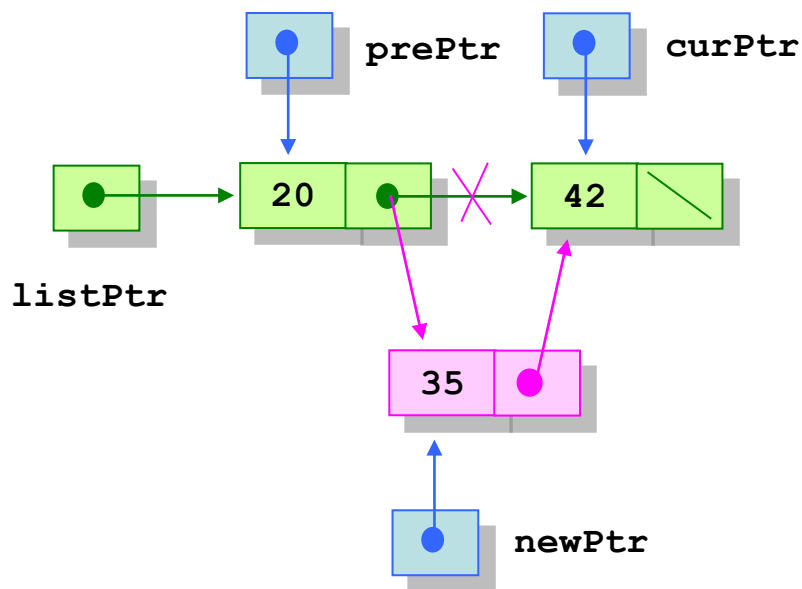
实现

```
        default:
            printf("Invalid choice.\n\n");
            menu();
            break;
    }
    printf("? ");
    scanf("%d", &choice);
}
printf("Bye-bye!\n");
}
```

案例分析：链表

链表

✿ 插入结点的函数



案例分析：链表

链表

✿ 插入结点的函数

```
void insert_node(ListNodePtr *sPtr, int data) {
    ListNodePtr newPtr, prePtr, curPtr;

    newPtr = malloc(sizeof(ListNode));

    if (newPtr != NULL) {
        newPtr->data = data;
        newPtr->nextPtr = NULL;

        prePtr = NULL;
        curPtr = *sPtr;

        while (curPtr != NULL && data > curPtr->data) {
            prePtr = curPtr;
            curPtr = curPtr->nextPtr;
        }
    }
```

案例分析：链表

链表

✿ 插入结点的函数

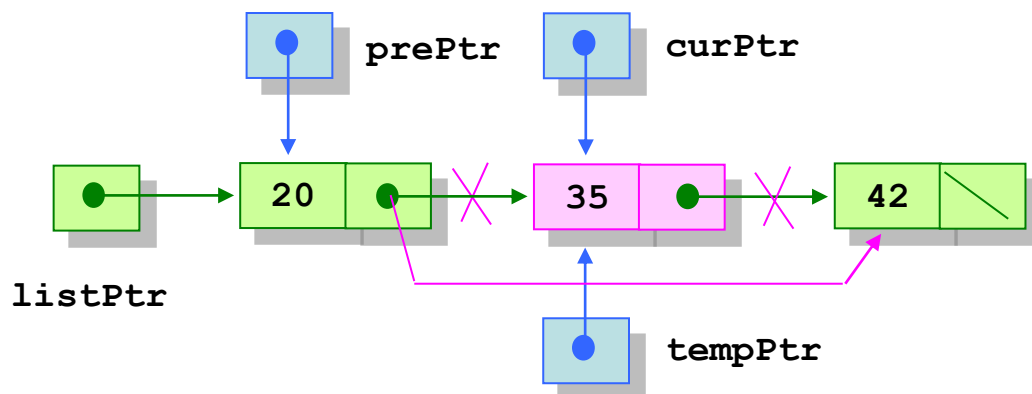
```
if (prePtr == NULL) {  
    newPtr->nextPtr = curPtr;  
    *sPtr = newPtr;  
}  
else {  
    prePtr->nextPtr = newPtr;  
    newPtr->nextPtr = curPtr;  
}  
}  
else  
    printf("%d not inserted. No memory available.\n", data);  
}
```

如果新结点是第一个节点，则必须调整链表的头指针

案例分析：链表

链表

✱ 删除结点的函数



案例分析：链表

链表

✿ 删除结点的函数

```
int delete_node(ListNodePtr *sPtr, int data) {  
    ListNodePtr prePtr, curPtr, tempPtr;  
  
    if (data == (*sPtr)->data) {  
        tempPtr = *sPtr;  
        *sPtr = (*sPtr)->nextPtr;  
        free(tempPtr);  
        return 1;  
    }  
    else {  
        prePtr = *sPtr;  
        curPtr = (*sPtr)->nextPtr;
```

如果删除的是第一个节点，必须调整链表的头指针

案例分析：链表

链表

✿ 删除结点的函数

```
while (curPtr != NULL && curPtr->data != data) {
    prePtr = curPtr;
    curPtr = curPtr->nextPtr;
}

if (curPtr != NULL) {
    tempPtr = curPtr;
    prePtr->nextPtr = curPtr->nextPtr;
    free(tempPtr);
    return 1;
}
return 0;
}
```

案例分析：链表

链表

✿ 判空和输出链表的函数

```
int is_empty(ListNodePtr listPtr) {  
    return listPtr == NULL;  
}  
  
void print_list(ListNodePtr curPtr) {  
    if (curPtr == NULL)  
        printf("List is Empty.\n\n");  
    else {  
        printf("The list is:\n");  
        while (curPtr != NULL) {  
            printf("%d --> ", curPtr->data);  
            curPtr = curPtr->nextPtr;  
        }  
        printf("NULL\n\n");  
    }  
}
```

小结

- 动态数据结构
- 自引用的结构体
- 动态内存分配
- 链表